

17 February 2026

Termination: how do we know it ends?

Either (i) runtime analysis

(ii) use a decrementing fn

→ proves that we do not wind up in infinite loops/recursion.

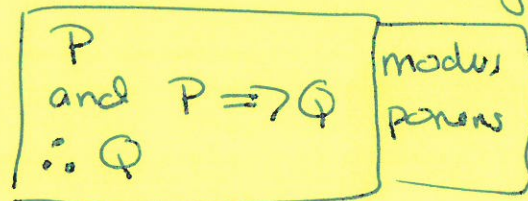
Correctness of an algorithm: Must prove

(A) That it terminates (see above)

(B) If it terminates, then it is correct
(this is called partial correctness.)

This is where loop/recursion invariants kick in.)

NOTE: By modus ponens, if we know (A) and (B), then we know that the algo is correct.



Sidenote: on reductions

- Problem: shortest path from a to b in a digraph

↑
Complexity of a problem: what is the best/fastest worst-case time for ANY algorithm to solve this problem?

Often, we have upper bounds $O(-)$
lower bounds $\Omega(-)$

But, tight bounds $\Theta(-)$ can be hard to come by.

Note: many open probs in CS are of the form: can I make $\Theta(-)$ smaller or $\Omega(-)$ larger?

- Algorithms: specific way to solve a prob.
We often know the tight runtime $\Theta(-)$
- Suppose we have prob $A \leftarrow$ asymptotics unknown
prob $B \leftarrow$ can be done in $\Theta(f(n))$

"reduction" Prob A to Prob B and we know one way of solving A is

- (i) reduce A to B in $\Theta(g(n))$ time
- (ii) solve B $\leftarrow$ known to be in $\Theta(f(n))$
- (iii) reverse sol'n to B to sol'n to A $\leftarrow$ $\Theta(g(n))$ time

$\Rightarrow$ this sol'n is $\Theta(g(n) + f(n))$
 $\Rightarrow$ the best sol'n is $\Theta(g(n) + f(n))$

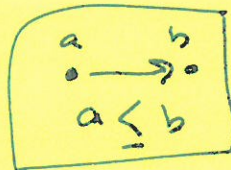
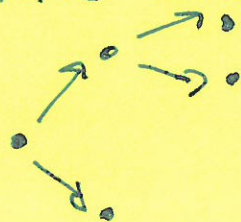
Well-ordered sets

$\mathbb{N}, \mathbb{Z}$

~~finite set~~

~~diagnose (finite)~~

finite subsets of $\mathbb{R}$



not well-ordered

$$(0,1) \leftarrow 0 \notin (0,1)$$

$I = [0,1] \leftarrow (0,1)$ does not have a minimum element.

Decrementing Fens:

$D: S \rightarrow \mathbb{N}$

$\uparrow$ set of all possible states — includes } $\uparrow$ our favorite codomain

Example:

coolalgo(x, y)
temp $\leftarrow$ x
temp += y
return y

} no loop, no recursion

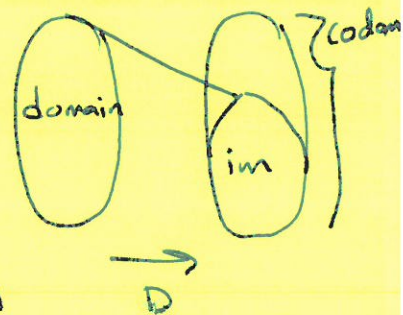
$D: S \rightarrow \mathbb{N}$
defined by
 $D(s) = 5 \quad \forall s \in S.$

signature of
fn
how to eval.
it

note: $0 = \min(\mathbb{N})$

but, 0 is not realized!

$\text{im}(D) = \{5\}$



Theor. Ex

$D: S \rightarrow \mathbb{N}$

Let s_i be the state encountered when entering the loop for the i th time.

Q: Why do decr. fens prove termination?

Practice

Provide decrementing functions for the following algorithms:

1. Suppose the algorithm has a single for loop, which appears as follows: `for i=10; i > 0; i--`. What is the decrementing function?
Try it!

Let $D_1: \mathcal{S} \rightarrow \mathbb{N}$ be defined by $\forall S \in \mathcal{S}$

$$D_1(S) = \begin{cases} i+1, & \text{if } i \text{ is defined}^* \\ 11, & \text{before loop} \\ 0, & \text{after the loop} \end{cases}$$

```

for()
| does something
end for
return
    
```

note:
 $D_1(S) = i$
also works

2. Write a for loop that iterates through an array A of length n starting at $A[1]$ and checking each index in order until $A[n]$, in order to find the maximum value in the array. Then, provide the decrementing function for that loop.
Try it!

$$D_2: \mathcal{S} \rightarrow \mathbb{N}$$

$$D_2(S) = n - i + 1$$

3. Suppose there is a perfect infinite neighborhood, where the post office is at $(0, 0)$ and for every $x, y \in \mathbb{N}$, there is a house at (x, y) . If you a person in this neighborhood can follow simple instructions (e.g., walk one block north/south/east/west), write a while loop that will give directions from a given house to the post office. Then, prove that your algorithm terminates.
Try it!

4. Bubble sort. (Note: there are two loops here! On first attempt, just worry about the loops independently and come up with two functions.)